

Tour Plan

Statement Explanation

Chef একটি roadtrip এ যাচ্ছে এবং cab fare calculate করতে চায়।

Cab operator-এর pricing system:

First 50 km পর্যন্ত একটি fixed cost = X টাকা।

যদি distance 50 km এর বেশি হয়, তাহলে extra প্রতিটা kilometer এর জন্য Y টাকা/km দিতে হবে।

মোট distance = Z km

তোমার কাজ → Chef-এর total payment বের করা।

এখানে মূল objective হচ্ছে distance 50 এর মধ্যে নাকি বেশি — এই condition handle করা।

Hint 1

প্রথমে check করো — Chef-এর destination কি 50 km এর বেশি?

কারণ 50 km পর্যন্ত সবসময় same fixed charge।

Hint 2

যদি $Z \leq 50$ হয়:

তাহলে extra distance নেই।

So answer হবে শুধু:

→ fixed cost = X

Hint 3

যদি $Z > 50$ হয়:

তাহলে দুইটা অংশে ভাবো—

First 50 km → cost = X

Remaining distance $\rightarrow (Z - 50)$ km

Hint 4

Extra distance-এর cost বের করো:

extra km $\times$ cost per km

তারপর fixed cost এর সাথে add করো।

Key Observation

এই problem-এর পুরো logic মাত্র one condition এর উপর দাঁড়িয়ে:

Distance within 50 $\rightarrow$ only fixed cost

Distance beyond 50 $\rightarrow$ fixed cost + extra distance cost

Easy Speaking

Statement Explanation

Chef মনে করে একটা word hard to pronounce যদি সেখানে কমপক্ষে 4টা consecutive (contiguous) consonant থাকে।

Consonant বলতে সব letters except:

a, e, i, o, u

মানে vowel ছাড়া সব character consonant।

তোমাকে একটা string S দেওয়া হবে। Check করতে হবে — word টা pronounce করা hard কিনা।

Output:

Yes $\rightarrow$ যদি 4 বা তার বেশি consecutive consonant থাকে

No → না থাকলে

Target

Goal হচ্ছে string-এর মধ্যে continuous consonant streak detect করা।

তোমার কাজ total consonant count বের করা না —

বরং একটানা কয়টা consonant এসেছে সেটা track করা।

Hint 1

String-এর প্রতিটা character একবার করে traverse করো।

প্রতিটি character এর জন্য check করো:

এটা vowel?

নাকি consonant?

Hint 2

একটা variable রাখো:

current_consonant_count

যখন consonant পাও:

→ count increase করো

যখন vowel পাও:

→ count আবার 0 করে দাও

কারণ consonant sequence break হয়ে গেছে।

Hint 3

Traverse করার সময় যদি কোনো মুহূর্তে:

count ≥ 4

হয়ে যায়—

তাহলে আর continue করার দরকার নেই।

Answer হবে:

Yes

Hint 4

খেয়াল করো—

এখানে পুরো word consonant হওয়া লাগবে না।

যেকোনো continuous substring যদি 4 consonant হয় তাহলেই enough।

Key Observation

এই problem-এর core idea:

Consonant → streak বাড়াও

Vowel → streak reset

Streak reaches 4 → hard to pronounce

Beginnings and Endings

Statement Explanation

একটা array good হবে যদি:

$$A[1] = A[N]$$

মানে first element এবং last element equal হতে হবে।

তুমি adjacent swap করতে পারো:

`swap(A[i], A[i+1])`

এক move এ শুধু পাশাপাশি দুইটা element swap করা যাবে।

তোমার target $\rightarrow$ minimum adjacent swaps করে array-কে good বানানো।

যদি কোনোভাবেই সম্ভব না হয় $\rightarrow$ print -1

Target

শেষে এমন একটা arrangement বানাতে হবে যেখানে:

`first element == last element`

আর swap count minimum রাখতে হবে।

এখানে পুরো array sort করা লাগবে না।

শুধু দুই endpoint equal করা enough।

Hint 1

কোন value শেষে first আর last position এ থাকবে?

খেয়াল করো—

যদি কোনো value পুরো array-তে মাত্র ১ বার থাকে, তাহলে সে কখনো first এবং last দুই জায়গায় একসাথে থাকতে পারবে না।

তাই প্রথম observation:

যদি সব element unique হয় $\rightarrow$ answer = -1

Hint 2

ধরো কোনো value x কমপক্ষে দুইবার আছে।

এখন question:

minimum swap এ কিভাবে x কে first/last এ আনা যায়?

Hint 3

Adjacent swap দিয়ে:

index i থেকে first position এ আনতে লাগবে $\rightarrow$ i-1 swaps

index i থেকে last position এ আনতে লাগবে $\rightarrow N-i$ swaps
(1-indexed চিন্তা করলে)

Hint 4

একই value-এর multiple occurrence থাকতে পারে।

ধরো value x positions এ আছে:

$p_1, p_2, p_3 \dots$

তখন এমন দুইটা occurrence choose করতে হবে—

একটা first এ যাবে

একটা last এ যাবে

এবং total movement minimum হবে।

Hint 5

Actually আরও simplify করা যায়।

কারণ endpoints ছাড়া middle arrangement matter করে না।

একটা occurrence কে left end এ টানা

আর আরেকটা occurrence কে right end এ টানা—

এই দুই movement-এর total minimize করো।

সব value-এর উপর try করো।

Key Observation

এই problem-এর main trick:

Array reorder না — endpoint equal করা।

Adjacent swaps-এর cost = distance travelled।

Magic Mirror

Statement Explanation

Chef দুইটা magic mirror বসিয়েছে positions:

$$X1 < X2$$

যখনই Chef কোনো box:

$$X1 - d$$

black করে, mirror-এর কারণে automatically আরেকটা box:

$$X2 + d$$

black হয়ে যায়।

মানে সবসময় box pair আকারে color হবে।

একটা operation এ তৈরি হওয়া pair:

$$(X1-d, X2+d)$$

এখন তোমাকে final black boxes দেওয়া আছে:

$$A1 < A2 < \dots < AN$$

Check করতে হবে—

এমন কোনো mirror position choose করা সম্ভব কিনা যাতে exactly এই black boxes পাওয়া যায়।

Target

মূল target:

দেওয়া black boxes-গুলোকে এমনভাবে pair করা যায় কিনা যেন প্রতিটা pair:

$$\text{left} + \text{right} = \text{same value}$$

কারণ:

$$(X1-d) + (X2+d) = X1 + X2$$

এখানে d cancel হয়ে যায়।

অর্থাৎ—

সব operation-এর pair-এর sum সবসময় constant থাকে।

Hint 1

প্রতিটা operation exactly 2টা box color করে।

তাই black boxes সবসময় pair আকারে exist করবে।

(Problem already guarantee করেছে N even.)

Hint 2

Mirror operation-এর সবচেয়ে important property:

$$(X1-d)+(X2+d)=X1+X2$$

Notice:

d disappear!

মানে সব generated pair-এর sum একই হবে।

Hint 3

যেহেতু array sorted:

$$A1 < A2 < \dots < AN$$

Try pairing:

$$(A1, AN)$$

$$(A2, A(N-1))$$

$$(A3, A(N-2))$$

কেন?

কারণ smallest left-এর সাথে কোনো valid matching হলে largest right-এর সাথে match হওয়া ছাড়া balance রাখা কঠিন।

Hint 4

প্রতিটা mirrored pair-এর জন্য check করো:

$$A[i] + A[N-i+1]$$

সবগুলোর sum কি equal?

Hint 5

যদি কোনো এক pair-এর sum অন্য pair থেকে different হয়—

তাহলে impossible।

কারণ একটা experiment-এ:

$$X1 + X2$$

একটাই fixed থাকে।

Key Observation

Mirror operation creates symmetric pairs.

For every colored pair:

$$\text{left} + \text{right} = \text{constant}$$

So entire problem reduces to:

Can the array be paired from both ends so that every pair has identical sum?

No simulation needed. Only observe the structure.

Planting Roses

Statement Explanation

তোমার কাছে N different color-এর rose আছে।

A_i = i-th color এর available rose সংখ্যা

Garden-এ মোট M টা pots আছে।

প্রতিটা pot:

empty থাকতে পারে

অথবা একটাই rose থাকতে পারে

কিন্তু দুইটা rule maintain করতে হবে:

Rule 1

Adjacent দুইটা filled pot-এ different color থাকা যাবে না।

মানে:

[1 1 1] ✓

[2 2 0 2] ✓

[1 2] ✗

Different color হলে মাঝে অন্তত একটা empty pot লাগবে।

Rule 2

কোনো জায়গায় K-এর বেশি consecutive filled pot থাকতে পারবে না।

মানে maximum continuous planted block size = K

Example (K=3):

[1 1 1 0 2 2] ✓

[1 1 1 1] ✗

Target → maximum কতগুলো rose plant করা যায়।

Target

Maximum planted roses বের করতে হবে।

কিন্তু তিনটা limitation আছে:

Available flowers (A_i)

Total pots (M)

Consecutive planted block limit (K)

Hint 1

Rule 1 থেকে একটা important observation:

Different color adjacent হতে পারবে না।

তাই প্রতিটা color একটা continuous block হিসেবে ভাবা যায়।

Example:

1 1 1 0 2 2 0 3

প্রতিটা color $\rightarrow$ আলাদা segment।

Hint 2

Rule 2 বলছে—

একটা segment-এর size K -এর বেশি হতে পারবে না।

তাহলে কোনো color থেকে actual ব্যবহার করা যাবে:

$\min(A_i, K)$

কারণ K -এর বেশি নিলেই invalid।

Hint 3

এখন ধরো আমরা কিছু colors choose করলাম।

প্রতিটা chosen color একটা block বানাবে।

দুই block-এর মাঝে কমপক্ষে ১টা empty pot লাগবে।

যদি c টা color ব্যবহার করো:

needed empty pots = $c - 1$

Hint 4

ধরো selected block sizes:

$b_1, b_2, b_3 \dots$

তাহলে total occupied pots হবে:

$(b_1 + b_2 + \dots) + (\text{number_of_blocks} - 1)$

এটা M exceed করতে পারবে না।

Hint 5

যেহেতু objective maximum roses—

তাই প্রতিটা color থেকে যত বেশি সম্ভব নেওয়া উচিত।

মানে আগে compute করো:

$\text{usable}_i = \min(A_i, K)$

তারপর decide করো কতগুলো block fit করানো যায়।

Hint 6

Observe করো—

একটা নতুন color add করলে:

কিছু rose যোগ হয়

কিন্তু extra একটা empty pot cost হয়

তাই সব color blindly নেওয়া optimal নাও হতে পারে।

Key Observation

Problem-টা আসলে flower placement না—

Think in terms of:

Color $\rightarrow$ one block

Block size $\leq K$

Between blocks $\rightarrow$ 1 empty pot

So maximize:

(total roses used)

subject to:

$\text{sum}(\text{block sizes}) + \text{gaps} \leq M$